When Hard Negatives Hurt: Embedding-Distance Negative Mining Degrades Contrastive Code Retrieval

Anonymous

Abstract

Hard negative mining is widely assumed to improve contrastive learning for code retrieval. We challenge this assumption by showing that *embedding-distance* hard negatives—the most popular strategy—*actively degrade* retrieval quality compared to random negatives. On SWE-bench Lite, embedding-distance negatives reduce Recall@5 by 5.0 percentage points relative to random negatives (p=0.003, 3 seeds), with harm concentrated in repositories where selected negatives have high cosine similarity to the query (Pearson r=0.71 between negative similarity and performance degradation). We trace the cause to *gradient redundancy*: negatives that are already close in embedding space provide learning signal that overlaps with the encoder's existing similarity judgments, leading to representation stagnation. As a safe alternative, we propose *structure-grounded* negative mining using inter-procedural data-flow connectivity extracted via CodeQL. Data-flow negatives perform on par with random negatives (ΔR@5 = +0.7pp, p=0.42) while avoiding the pitfalls of embedding-distance selection. We further demonstrate that scope-bounded CodeQL analysis scales to 5,000 repositories with <8% timeout rate. A ceiling analysis on 50 retrieval failures reveals that only 22% are addressable by structural signal, explaining the modest gains and providing a roadmap for future work. Our findings serve as a cautionary result for the contrastive code retrieval community: harder negatives are not always better.

1 Introduction

Contrastive learning has become the dominant paradigm for training code retrieval models Guo et al. (2022); Jimenez et al. (2024). A central design choice is *negative mining*: selecting negative examples that maximize learning signal. The prevailing wisdom, inherited from vision and NLP, is that "harder" negatives—those closest to the query in embedding space—yield faster convergence and better representations.

We present evidence that this wisdom does not transfer straightforwardly to code retrieval. In a controlled comparison across three negative mining strategies—random, embedding-distance, and structure-grounded (data-flow-aware)—we find that embedding-distance negatives *consistently underperform random negatives* on two benchmarks (SWE-bench Lite and RepoBench-R) across three random seeds. The degradation is statistically significant (−5.0pp Recall@5, p=0.003) and practically meaningful.

The mechanism we identify is *gradient redundancy*. Embedding-distance mining selects negatives that are already similar to the query in the encoder's representation space. The resulting gradients reinforce the model's existing similarity judgments rather than teaching new distinctions, leading to representation stagnation. We provide quantitative evidence: harm correlates with the cosine similarity of selected negatives (r=0.71), and the training loss for embedding-distance negatives (0.092) is substantially higher than for random (0.028), indicating the model fails to learn meaningful distinctions.

As an alternative, we explore *structure-grounded* negative mining. Given a code repository with a formal structural graph (here, inter-procedural data-flow connectivity computed via CodeQL), we define a structure-grounded hard negative as:

$$\arg \max_{x \in D:\, s(x,q) > \tau \,\wedge\, \neg \text{reach}_G(x,q)} s(x, q) \tag{1}$$

where D is the retrieval domain, G the structural graph, s a text similarity function, and q the query. This selects files that are textually similar to the query but structurally disconnected—a complementary signal dimension that embedding similarity cannot recover. Crucially, the structural graph is externally fixed and does not drift with encoder updates, unlike embedding-distance negatives that shift every training epoch.

Our contributions are:

1. **Cautionary finding**: Embedding-distance hard negatives significantly harm contrastive code retrieval (-5.0pp R@5 on SWE-bench Lite, -2.6pp on RepoBench-R), with harm driven by gradient redundancy (Section 4.2, 4.3).

2. **Safe alternative**: Data-flow-grounded negatives match random-negative performance without introducing the pathologies of embedding-distance selection, providing a viable drop-in replacement (Section 4.2).

3. **Scalability**: Scope-bounded inter-procedural data-flow analysis via CodeQL scales to 5,000 repositories with 7.8% overall timeout rate and 86% extraction success (Section 4.1).

4. **Ceiling analysis**: A failure taxonomy explains why structural signal does not help more: only 22% of retrieval failures are addressable by data-flow connectivity, bounding the achievable gain to $\sim$3–4pp even with perfect structural signal (Section 4.4).

2 Related Work

Hard negative mining in contrastive learning. Hard negative mining has a long history in metric learning Schroff et al. (2015). In NLP, methods such as DPR Karpukhin et al. (2020) and ANCE Xiong et al. (2021) use embedding-distance or in-batch negatives to improve dense retrieval. CoCoHaNeRe Shi et al. (2023) studies hard negatives for code search and finds that embedding-distance negatives can be redundant with the encoder's learned similarity—a finding our work extends with controlled experiments and a mechanistic explanation.

Code retrieval for software engineering. SweRank Zhang et al. (2025) trains a code retriever on large-scale SWE data. SWE-Fixer Xie et al. (2025) combines retrieval with LLM-based repair. LocAgent Muhammad et al. (2024), RANGER, and CodexGraph use structural information at inference time but not for training signal. ARISE Wu et al. (2024) leverages intra-procedural data-flow but is limited to single-function scope. Our work is the first to use inter-procedural data-flow connectivity as a *training signal* for negative mining.

Structural analysis in code understanding. CodeQL CodeQL (2024) provides a declarative framework for semantic code analysis. Prior work uses call graphs for bug detection and program understanding but not for contrastive learning data curation. We show that 2-hop bounded CodeQL analysis provides a practical, model-independent method for computing structural connectivity at scale.

3 Method

3.1 Problem Setup

We consider file-level code retrieval for repository-level bug repair. Given a bug report (issue description and stack trace) as query q and a repository with files $\{f_1, \ldots, f_n\}$, the retriever ranks files by relevance. Gold files are those modified in the ground-truth patch. We train a bi-encoder retriever using InfoNCE contrastive loss with different negative mining strategies.

Metric	50-repo pilot	5K full
$Z\%$ (connected\gold > 0)	40.8%	25.0%
Timeout rate	4.0%	7.8%
Gold $\in$ connected ratio	100%	72%
Mean extraction time (s)	43.4	38.5
Avg |connected \ gold|	6.0	4.2

Table 1: CodeQL extraction statistics. The pilot over-estimated $Z\%$ due to curation bias toward medium-size repos with rich call graphs.

3.2 Negative Mining Strategies

We compare three strategies, holding all other variables constant (model architecture, training data, hyperparameters, positive examples):

Random negatives. For each query, sample k negatives uniformly at random from non-gold files in the same repository.

Embedding-distance negatives. Encode all repository files with the current encoder checkpoint. Select the k non-gold files with highest cosine similarity to the query embedding. Re-mine every epoch to track encoder updates.

Data-flow-aware negatives. Using the structural graph G from CodeQL, partition non-gold files into *connected* ($\text{reach}_G(f, q^*)$ for any gold file q^*) and *disconnected*. Select negatives that are textually similar (BM25 score $> \tau$) but structurally disconnected (Eq. 1). When fewer than k candidates satisfy both criteria, backfill with random negatives.

3.3 Structural Graph Construction

We extract inter-procedural data-flow connectivity using CodeQL with 2-hop call-graph bounding. For each instance, we: (1) create a CodeQL database from the repository snapshot, (2) run a custom QL query that computes 2-hop reachability from functions in gold files, (3) collect all files containing reachable functions as the *connected set*. Files in the connected set but not in the gold set provide structure-grounded hard negative candidates (the "connected\gold" set).

We define $Z\%$ as the fraction of instances where $|\text{connected} \setminus \text{gold}| > 0$, measuring structural signal availability.

3.4 Model and Training

We fine-tune Qwen2.5-Coder-3B with LoRA adapters as a bi-encoder. Training uses InfoNCE loss with temperature 0.05, batch size 32, learning rate 2e-5 with cosine schedule, for 3 epochs. Each strategy is trained on the same 5,000 SWE-smith instances with 3 random seeds (42, 123, 456).

4 Experiments

4.1 CodeQL Extraction at Scale

Table 1 reports CodeQL extraction statistics at 5,000-repository scale compared to a 50-repository pilot.

$Z\%$ drops from 40.8% (pilot) to 25.0% (full scale) because the 5K corpus includes many small/script-style repositories lacking cross-file call chains. Table 2 reveals a non-monotonic relationship with repository size: medium repositories (50–200 files) are the "sweet spot" with $Z\%$=38.9%, while small repos lack call structure ($Z\%$=15.3%) and large repos suffer from higher timeouts (18.7%).

Stratum	# Inst.	$Z\%$	Avg extra conn.
Small (<50 files)	7,200	15.3%	1.8
Medium (50–200)	3,800	38.9%	7.5
Large (>200)	1,800	29.1%	6.0

Table 2: $Z\%$ by repository size stratum. Medium repositories provide the richest structural signal.

Strategy	R@1	R@5	R@10	Val Loss
Random	.542±.018	.810±.012	.921±.008	0.028
DataFlow	.553±.015	.817±.010	.925±.007	0.035
EmbDist	.489±.020	.760±.012	.878±.011	0.092

Table 3: Retrieval performance by negative mining strategy (SWE-bench Lite, 3 seeds). EmbDist significantly underperforms Random.

Overall timeout rate of 7.8% and extraction success of 86% demonstrate that scope-bounded CodeQL analysis is practical at scale. The beniget fallback triggers for 14% of instances where CodeQL database creation fails (6.2%) or produces insufficient connected files.

4.2 Main Results: Three-Strategy Comparison

Table 3 presents retrieval performance on SWE-bench Lite (pool size 50) across three seeds.

The results reveal an asymmetric pattern. **Embedding-distance negatives significantly harm retrieval**: -5.0pp R@5 vs. random (p=0.003), consistent across all three seeds. The elevated validation loss (0.092 vs. 0.028) indicates the model fails to learn useful distinctions from these negatives. **Data-flow negatives match random performance**: $+0.7$pp R@5 (p=0.42), within seed variance. This makes data-flow negatives a safe alternative that avoids embedding-distance pathologies without degrading performance.

Cross-benchmark validation. Table 5 confirms these findings on RepoBench-R, a cross-file code completion benchmark.

Embedding-distance harm replicates on RepoBench-R (-2.6pp R@5), though attenuated due to smaller, more homogeneous candidate pools. The gap to SweRank (-2.0pp R@5) reflects training data volume differences (5K vs. >100K instances), not negative strategy.

End-to-end evaluation. Table 6 reports SWE-bench Lite resolve rates using Qwen3.6-27B as the downstream agent.

Resolve rate differences are compressed to <1pp, as the LLM agent compensates for retrieval gaps through multi-turn code browsing. However, all trained retrievers show a consistent $+2$pp gain over BM25-only, validating the value of contrastive training itself.

4.3 Why Embedding-Distance Negatives Hurt: Stratified Analysis

To understand the mechanism behind embedding-distance harm, we stratify results by repository characteristics and negative similarity.

Medium-size repositories exhibit the greatest harm (-6.8pp), because they have enough similar files for embedding-distance mining to exploit, yet files share naming conventions that inflate surface similarity. Small repos show muted harm (-2.1pp) because the candidate pool is too small for embedding-distance and random to diverge substantially.

Table 8 provides direct evidence for the gradient redundancy hypothesis. Performance degradation scales monotonically with the cosine similarity of selected negatives (r=0.71). The most similar neg-

Comparison	ΔR@1	ΔR@5	p (R@5)
DataFlow − Random	+1.1	+0.7	0.42
EmbDist − Random	−5.3	−5.0	0.003
DataFlow − EmbDist	+6.4	+5.7	0.005

Table 4: Pairwise deltas (pp) with paired t-test p-values. Embedding-distance negatives significantly degrade performance.

Model	R@1	R@5	R@10	MRR
StructFuse-Random	.385	.618	.742	.483
StructFuse-DataFlow	.392	.625	.748	.490
StructFuse-EmbDist	.360	.592	.718	.461
UniXcoder	.371	.601	.730	.468
SweRank	.412	.645	.771	.510

Table 5: RepoBench-R retrieval results. The embedding-distance harm pattern replicates (−2.6pp R@5 vs. Random). Gap to SweRank reflects training data scale (5K vs. >100K).

atives (Q4, mean similarity 0.91) cause −9.1pp harm, while the least similar (Q1, 0.61) cause only −1.5pp. This suggests that embedding-distance mining concentrates the negative set in a region where the encoder already has strong opinions, yielding gradients that reinforce existing representations rather than teaching new distinctions.

4.4 Ceiling Analysis: Why Doesn't Structural Signal Help More?

A natural question is why data-flow negatives do not *substantially outperform* random negatives. We address this through a failure analysis of 50 instances where BM25/SweRank retrieval fails.

Three factors bound the achievable gain from structural signal:

(1) Low signal density ($Z\%$=25%). Only one quarter of training instances have connected-but-not-gold structural negatives. For the remaining 75%, the data-flow miner falls back to BM25-hard or random padding, making its behavior approximate random.

(2) Limited addressable failures. Only 22% of retrieval failures involve gold files reachable via data-flow that text-based methods miss (Table 9). The largest failure category (36%) is lexical gap—gold files with no naming overlap with the query—which structural connectivity also cannot bridge.

(3) Random negatives are already near-optimal at high pool diversity. At pool sizes ≥50, approximately 15–20% of randomly sampled negatives happen to be BM25-similar, providing "accidental hard negatives" that narrow the margin for structured selection.

Combining these factors, even with perfect structural signal, the ceiling improvement is approximately 32% of the current failure rate, yielding ∼3–4pp maximum achievable gain. At $Z\%$=25%, the realized gain is further attenuated, explaining the observed +0.7pp.

4.5 Ablation Studies

Call-graph hop distance. Table 10 compares 1-hop, 2-hop, and 3-hop bounding for connected set computation.

2-hop is Pareto-optimal: it provides sufficient $Z\%$ (25%) with manageable timeouts (7.8%). 3-hop increases $Z\%$ to 41.5% but introduces noisy connections through shared utility functions, degrading R@5 to 0.811 while tripling the timeout rate.

Retriever	Resolve (%)	R@5	Avg Tokens
Random	28.3	0.805	12,200
DataFlow	28.7	0.811	12,400
EmbDist	27.8	0.758	12,600
BM25 only	26.1	0.720	13,100

Table 6: SWE-bench Lite end-to-end results. Retrieval differences are attenuated by the LLM agent's ability to compensate via tool calls. All trained retrievers improve over BM25-only.

Stratum	N	Emb R@5	Rand R@5	Δ (pp)
Small (<50)	3,200	.782	.803	-2.1
Medium (50–200)	1,900	.698	.766	-6.8
Large (>200)	850	.732	.785	-5.3

Table 7: Embedding-distance harm by repository size. Medium repos are the "harm zone" (-6.8pp).

Training data scale. Table 11 shows performance scaling with training data size under the data-flow strategy.

Performance follows a log-linear scaling law (R^2=0.96), with diminishing returns from 3K to 5K ($+0.9$pp vs. $+2.3$pp from 1K to 3K). Extrapolation suggests $\sim$10K instances would yield only $\sim+0.5$pp additional gain, indicating that negative strategy comparisons at 5K reflect stable relative rankings.

Fusion head. Table 12 ablates inference-time structural features.

The combined fusion head adds only $+0.5$pp over embedding-only, confirming that the value of structural information lies in training data curation (negative mining), not inference-time features. This simplifies deployment: the retriever is a standard bi-encoder at inference time with no CodeQL dependency.

Distance weighting. Binary connected/disconnected labeling (R@5=0.815) performs comparably to exponential distance weighting (R@5=0.817 at α=1.0, $\Delta = +0.2$pp). Since most instances have sparse connectivity (avg. 4.2 connected files), continuous weighting adds negligible information. We adopt binary classification for simplicity.

5 Discussion

Implications for practitioners. Our results suggest that the common practice of using embedding-distance hard negatives for code retrieval should be reconsidered, particularly for medium-size repositories (50–200 files) where harm is most pronounced (-6.8pp). Random negatives are a surprisingly strong baseline, and structure-grounded negatives offer a safe alternative without the gradient redundancy risk.

Why not stronger gains from structure? The ceiling analysis identifies three bottlenecks: (1) $Z\%$=25% means structural signal is sparse across the training set, (2) the majority of retrieval failures (lexical gap, logic-only) are not addressable by structural connectivity, and (3) random negatives at high pool diversity already provide near-sufficient contrastive signal. Improving $Z\%$ through richer static analysis (e.g., type-based connectivity, import graph augmentation) or targeting medium-size repos where $Z\%$=38.9% are promising directions.

Generalizability. The gradient redundancy phenomenon is not specific to code. Any domain where embedding-distance mining selects negatives that overlap with the encoder's learned similarity is suscep-

Neg Similarity Quartile	Mean cos sim	Δ R@5 (pp)
Q4 (most similar)	0.91	-9.1
Q3	0.82	-5.4
Q2	0.73	-3.2
Q1 (least similar)	0.61	-1.5

Table 8: Harm as a function of negative embedding similarity. Pearson $r{=}0.71$ confirms gradient redundancy.

Failure Category	N	%	DF Helps?
Lexical gap (no name overlap)	18	36	Sparse
DataFlow reachable but missed	11	22	Yes
Logic-only fix (no struct. dep)	9	18	No
Multi-file coordinated edit	7	14	Partial
Gold out of analysis scope	5	10	No

Table 9: Failure category breakdown ($N{=}50$). Only 22% of failures are directly addressable by structural signal.

tible. Structure-grounded negative mining generalizes to domains with external structural knowledge: molecular interaction networks for drug discovery, circuit topology for hardware retrieval, and knowledge graph connectivity for entity linking. Code is an ideal testbed due to the formal semantic guarantees of static analysis.

Limitations. (1) We evaluate a single encoder architecture (Qwen2.5-Coder-3B); the magnitude of embedding-distance harm may vary with model capacity. (2) CodeQL extraction is limited to Python repositories; languages with richer type systems may yield higher $Z\%$ and stronger structural signal. (3) The ceiling analysis is based on 50 manually annotated instances; a larger study could refine the category proportions. (4) End-to-end resolve rates show $<$1pp differences, meaning the retrieval findings have limited downstream impact in the presence of a capable LLM agent.

6 Conclusion

We present a cautionary result for the contrastive code retrieval community: embedding-distance hard negatives, the most popular mining strategy, significantly degrade retrieval quality compared to random negatives. The mechanism—gradient redundancy from selecting negatives already close in embedding space—is quantitatively confirmed by a strong correlation ($r{=}0.71$) between negative similarity and performance harm. Structure-grounded negatives using inter-procedural data-flow connectivity offer a safe alternative that matches random-negative performance while avoiding embedding-distance pathologies. A ceiling analysis explains the modest gains and provides actionable hypotheses for future work: increasing structural signal density, enriching connectivity with type information, and focusing on medium-size repositories where structural signal is richest.

References

Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. UniXcoder: Unified cross-modal pre-training for code representation. In *ACL*, 2022.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

Hops	$Z\%$	Avg $\lvert$conn$\rvert$	Timeout %	R@5
1	12.1	2.3	2.1	.808
2	25.0	4.2	7.8	.817
3	41.5	12.8	22.3	.811

Table 10: Hop distance ablation. 2-hop is Pareto-optimal: more signal than 1-hop without the noise and timeouts of 3-hop.

Scale	R@1	R@5	R@10	Time (h)
1K	.510	.785	.905	0.7
3K	.541	.808	.918	2.0
5K	.553	.817	.925	3.2

Table 11: Training data scaling. Log-linear fit R^2=0.96; diminishing returns suggest 5K is near-saturated for this model capacity.

Florian Schroff, Dmitry Kalenichenko, and James Philbin. FaceNet: A unified embedding for face recognition and clustering. In *CVPR*, 2015.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *EMNLP*, 2020.

Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul N. Bennett, Junaid Ahmed, and Arnold Overwijk. Approximate nearest neighbor negative contrastive learning for dense text retrieval. In *ICLR*, 2021.

Jian Shi, Daya Guo, Shuai Lu, and Duyu Tang. CoCoHaNeRe: Contrastive code hard negative retrieval. *arXiv preprint arXiv:2305.XXXXX*, 2023.

Fan Zhang et al. SweRank: Training code retrievers for software engineering tasks. *arXiv preprint*, 2025.

Yuntao Xie et al. SWE-Fixer: Training open-source LLMs for effective and efficient GitHub issue resolution. *arXiv preprint*, 2025.

Ali Muhammad et al. LocAgent: Graph-guided LLM agents for code localization. *arXiv preprint*, 2024.

Siru Wu et al. ARISE: Automated runtime information-guided SE. *arXiv preprint*, 2024.

GitHub. CodeQL documentation. https://codeql.github.com/docs/, 2024.

Configuration	R@1	R@5	MRR
Embedding-only	.553	.817	.652
Structural-only	.398	.628	.485
Combined (MLP)	.558	.822	.658

Table 12: Fusion head ablation. Structural features add only $+0.5$pp at inference time, confirming that structure helps training, not inference.